

Phase 1

RISC-V 32I Assembly and Assembler

EEL 4768 Fall 2026

Introduction

This phase consists of two parts designed to develop your understanding of the RISC-V instruction set and assembly programming. In the first part, you will review assembly programming and gain hands-on experience with the 32-bit RISC-V (RV32I) instruction set architecture (ISA) used throughout this course. You are expected to have prior experience writing and debugging C and assembly programs.

In the second part, you will apply this knowledge by implementing your own assembler. Your assembler will translate RISC-V assembly programs into their corresponding binary machine-code representations.

For both parts, you may use **only instructions from the base RV32I ISA**. The RISC-V RV32I reference card provided on Webcourses may be used as a reference. Programs must use `ecall` to terminate execution correctly; examples demonstrating its use will be provided. Use of any instruction outside the base RV32I instruction set will result in a **zero for the corresponding problem**.

Submission

Only **one submission per group** is required.

Gradescope allows multiple students to be associated with a single submission through **“View or Edit Group.”** Ensure that your Gradescope group matches your assigned group on Webcourses.



Students who are not included in the submission, or who are added to a group they are not a member of, will **not receive credit for the submission**.

Grading Rubric

RISC-V Assembly	Multiplication	0.5
	GEMM	0.5
	Sobel	0.5
		1.5
Assembler	Multiplication	0.166
	GEMM	0.166
	Sobel	0.166
	Instruction Test	1
		1.5
Total:		3

Part 1: RISC-V Assembly

In this part, you will write RISC-V assembly programs to solve a series of problems listed below. A skeleton file for each problem is provided, along with an example program, `addition.s`, that performs a simple addition operation. The example demonstrates the basic structure of a RISC-V assembly program and how to properly terminate execution.

Multiplication:

The first problem is to multiply two numbers, stored in the words `a` and `b`, and store the result in the word `c`.

$$C = A \times B$$

Because the base RV32I ISA does not include dedicated multiplication instructions, you must implement multiplication using only the instructions provided. One approach is bit-serial multiplication, which computes the multiplication of each bit through shift instructions.

$$C = \sum_{i=0}^{N-1} B_i(A \ll i)$$

General Matrix Multiplication (GEMM):

The second problem requires you to compute the General Matrix-Matrix Multiplication (GEMM) of two 4×4 matrices, stored in the words a and b, and store the resulting 4×4 matrix in the word c.

The GEMM operation is defined as:

$$\begin{bmatrix} C_{11} & \cdots & C_{1j} \\ \vdots & \ddots & \vdots \\ C_{i1} & \cdots & C_{ij} \end{bmatrix} = \begin{bmatrix} A_{11} & \cdots & A_{1k} \\ \vdots & \ddots & \vdots \\ A_{i1} & \cdots & A_{ik} \end{bmatrix} \times \begin{bmatrix} B_{11} & \cdots & B_{1j} \\ \vdots & \ddots & \vdots \\ B_{k1} & \cdots & B_{kj} \end{bmatrix}$$
$$C_{ij} = \sum_{k=0}^{m-1} A_{ik} \times B_{kj}$$

The following C code illustrates the required loop ordering for computing GEMM. Given matrices (A), (B), and (C) with dimensions ([M,K]), ([K,N]), and ([M,N]), respectively, each element of (C) is computed by accumulating the products of the corresponding elements of (A) and (B).

```
for(m=0; m<M; m++){
    for(n=0; n<N; n++){
        for(k=0; k<K; k++){
            C[m,n] += A[m,k] * B[k,n];
        }
    }
}
```

Sobel Filter:

The Sobel filter is a commonly used image-processing technique for detecting edges by approximating the horizontal and vertical intensity gradients of an image. The filter computes these gradients by convolving the source image with two (3x3) kernels, (G_x) and (G_y), which capture changes in the horizontal and vertical directions, respectively.

The two kernels are defined as follows:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$$

$$G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Computing the convolution of a (3x3) kernel over a (5x5) image with a stride of 1 involves sliding the kernel across the source image and computing the element-wise multiplication and sum (reduction) at each position. This produces a (3x3) output matrix.

For example, given a (5x5) source matrix and a (3x3) kernel as shown below:

$$A = \begin{bmatrix} 4 & 5 & 1 & 2 & 3 \\ 3 & 0 & 1 & 3 & 5 \\ 4 & 6 & 7 & 3 & 1 \\ 2 & 1 & 0 & 8 & 9 \\ 3 & 6 & 6 & 3 & 4 \end{bmatrix} \quad B = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

1. First start by taking the first 3x3 window of matrix A to multiply against matrix B.

$$A_{w0} = \begin{bmatrix} 4 & 5 & 1 \\ 3 & 0 & 1 \\ 4 & 6 & 7 \end{bmatrix} \quad B = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

2. Then, multiply the window of A with kernel B where the indexes align.

$$\begin{aligned} 4 \times -1 &= -4 \\ 5 \times -2 &= -10 \\ 1 \times -1 &= -1 \\ 3 \times 0 &= 0 \\ 0 \times 0 &= 0 \\ 1 \times 0 &= 0 \\ 4 \times 1 &= 4 \\ 6 \times 2 &= 12 \\ 7 \times 1 &= 7 \end{aligned}$$

3. Finally, summate all the values to get the first result of the output matrix.

$$-4 + 10 + 1 + 0 + 0 + 0 + 0 + 12 + 7 = 4$$

$$C = \begin{bmatrix} 4 \\ \vdots \\ \vdots \end{bmatrix}$$

4. Then move right by the stride, which is one in this case, resulting in the next window being,

$$A_{w0} = \begin{bmatrix} 5 & 1 & 2 \\ 0 & 1 & 3 \\ 6 & 7 & 3 \end{bmatrix}$$

After moving the kernel across the entire row, return to the leftmost position and shift the kernel down by one row. Continue this process until the kernel has traversed the entire image. For a (5x5) image and a (3x3) kernel with a stride of 1, this produces a (3x3) output matrix.

For the Sobel filter, this convolution must be performed independently using both (G_x) and (G_y). The resulting matrices are then combined by adding their corresponding elements to produce a single (3x3) output matrix.

$$C = \begin{bmatrix} gx_{0,0} + gy_{0,0} & \cdots & gx_{0,3} + gy_{0,3} \\ \vdots & \ddots & \vdots \\ gx_{3,0} + gy_{3,0} & \cdots & gx_{3,3} + gy_{3,3} \end{bmatrix}$$

A visualization of a convolution can be found [here](#).

Part 2: RISC-V Assembler

In this part, you will implement an assembler for the **32-bit RISC-V (RV32I) ISA**, including support for the `ecall` instruction. Your assembler will translate a RISC-V assembly source file into separate output files containing the program's data and instructions in both hexadecimal and binary formats.

For example, given the input file `addition.s`, your assembler should generate the following files:

ASSEMBLY FILE	OUTPUT FILE	DESCRIPTION	EXAMPLE OUTPUT (NOT REAL VALUES)
ADDITION.S	addition_data.hex.txt	The data encoded as hex values, 1 byte per line.	0xA1 0x67 ... 0x5C
	addition_data.bin.txt	The data encoded as binary values, 1 byte per line.	10100001 01100111 ... 01011100
	addition_instr.hex.txt	The instructions encoded as hex values, 1 entry per line.	0xA1 0x67 ... 0x5C
	addition_instr.bin.txt	The instructions encoded as binary values, 1 byte per line	10100001 01100111 ... 01011100

Additionally, your assembler must use the following fixed memory addresses:

- **Instructions:** 0x00400000
- **Data:** 0x10010000

The assembler must be implemented in **Python** using the provided skeleton file as the main program. You can, and should, add to the main function, but do not remove what is provided.